

Domain Map

Mapa por domínio de negócio. Pense “vou mexer em Estoque” e encontre tudo aqui.

Diferente do REPO_MAP (organizado por pasta) e do FEATURE_TO_FILE_MAP (tabela), este documento agrupa por domínio funcional completo: frontend + backend + banco + integrações.

PEDIDOS

O fluxo mais crítico da Fluxa. Do carrinho ao KDS.

Camada	Arquivos
Frontend	pages/Menu.tsx , pages/MyOrders.tsx , contexts/CartContext.tsx
Backend	routers/orders.ts , routers/offlineOrders.ts
Banco	orders , orderItems , offlineOrders
Integra	Mercado Pago (pagamento), Dispatch (routing), Push (notificação)
Syntropy	Smart Routing, Forecast de demanda
Mission Control	Pedidos/min, ticket médio, top produtos

PAGAMENTOS

Pix, cartão, webhooks, confirmação.

Camada	Arquivos
Frontend	<code>components/StripePaymentForm.tsx</code> , <code>components/MercadoPagoCardForm.tsx</code> , <code>components/ExpressCheckout.tsx</code>
Backend	<code>routers/orders.ts</code> (create), <code>routers/stripe.ts</code> , <code>mercadopago.ts</code> , <code>_core/index.ts</code> (webhooks)
Banco	<code>orders</code> (status, paymentId, paymentMethod)
Integra	Mercado Pago API, Stripe API, Webhooks
Syntropy	Detecção de fraude, alertas de falha
Mission Control	Faturamento ao vivo, métodos de pagamento

ESTOQUE

Produtos, insumos, ficha técnica, movimentações.

Camada	Arquivos
Frontend	<code>pages/AdminStockMonitor.tsx</code> , <code>pages/AdminIngredients.tsx</code> , <code>pages/AdminProducts.tsx</code>
Backend	<code>routers/ingredients.ts</code> , <code>routers/products.ts</code>
Banco	<code>products</code> , <code>ingredients</code> , <code>productIngredients</code> , <code>stockMovements</code> , <code>ingredientMovements</code> , <code>ingredientTemplates</code> , <code>ingredientTemplateItems</code>
Integra	Débito automático no pedido (orders.ts)
Syntropy	Forecast de ruptura, alertas proativos, sugestão de reposição
Mission Control	Estoque ao vivo, alertas de ruptura

INGRESSOS

Tipos, compra, QR, check-in.

Camada	Arquivos
Frontend	<code>pages/TicketPurchase.tsx</code> , <code>pages/MyTickets.tsx</code> , <code>pages/AdminTickets.tsx</code> , <code>pages/AdminCheckIn.tsx</code>
Backend	<code>routers/tickets.ts</code>
Banco	<code>ticketTypes</code> , <code>ticketPurchases</code> , <code>eventEntries</code>
Integra	Mercado Pago (pagamento), QR Code (validação)
Syntropy	Previsão de público baseada em vendas
Mission Control	Ingressos vendidos, check-ins ao vivo

RETIRADA

QR de retirada, validação no bar, confirmação.

Camada	Arquivos
Frontend	<code>pages/Withdrawal.tsx</code> , <code>pages/StaffWithdrawal.tsx</code> , <code>components/AnimatedQR.tsx</code> , <code>components/QrScanner.tsx</code>
Backend	<code>routers/withdrawal.ts</code>
Banco	<code>withdrawalTokens</code>
Integra	Push (notificação de pronto)
Syntropy	—
Mission Control	Taxa de retirada, tempo médio

KDS (Kitchen Display System)

Tela do bar, pedidos para preparar, status.

Camada	Arquivos
Frontend	pages/StaffConsole.tsx , pages/StaffKDS.tsx , pages/StationKDS.tsx
Backend	routers/stationTokens.ts , routers/orders.ts (updateStatus)
Banco	stationTokens , salePoints
Integra	Dispatch (Smart Routing), Push (novo pedido)
Syntropy	Smart Routing (qual bar recebe qual pedido)
Mission Control	Filas por bar, tempo de preparo

FINANCEIRO

Split, repasse, faturas, receita da plataforma.

Camada	Arquivos
Frontend	pages/AdminFinancials.tsx , pages/AdminSplit.tsx , pages/AdminRecebimentos.tsx
Backend	routers/stripeConnect.ts , routers/fluxaControl.ts (financeiro)
Banco	splitConfigs , splitTransactions , platformRevenue , eventInvoices , platformFixedCosts , platformTaxConfig
Integra	Stripe Connect (split automático)
Syntropy	Projeção de faturamento, análise de margem
Mission Control	Faturamento, receita projetada, split

SYNTROPY (IA)

Inteligência, alertas, sugestões, ações automáticas.

Camada	Arquivos
Frontend	pages/AdminIntelligence.tsx , pages/AdminPostEvent.tsx , components/SyntropyLiveFeed.tsx , components/Full7BlocksLayer.tsx
Backend	intelligence-engine.ts , dispatch.ts , scheduled-ia-analysis.ts , routers/aiInsights.ts , routers/intelligence.ts
Banco	iaBaselines , aiInsights , iaActions , iaAnalysisLog , platformIntelligence , agentLogs
Integra	LLM (invokeLLM), Weather API, Push
Mission Control	Feed ao vivo, alertas, sugestões, ações

MISSION CONTROL

Dashboard ao vivo do evento.

Camada	Arquivos
Frontend	pages/MissionControl.tsx , components/mission-control/*
Backend	routers/fluxaControl.ts
Banco	Lê de: orders , products , salePoints , iaBaselines , aiInsights
Integra	Intelligence Engine (dados ao vivo)
Syntropy	Todas as engines alimentam o MC

CRM / PARTICIPANTES

Perfis, histórico, feedback, comunicação.

Camada	Arquivos
Frontend	<code>pages/AdminCRM.tsx</code>
Backend	<code>routers/fluxaControl.ts</code> (CRM section)
Banco	<code>participantProfiles</code> , <code>participantFeedback</code> , <code>leads</code>
Integra	Push (comunicação pós-evento)
Syntropy	Segmentação, perfil de consumo
Mission Control	Clientes ao vivo, ticket médio por pessoa

PROMOÇÕES

Promoções automáticas e manuais.

Camada	Arquivos
Frontend	<code>pages/AdminPromos.tsx</code>
Backend	<code>routers/fluxaControl.ts</code> (promos section)
Banco	<code>promos</code> , <code>autoConsumptionRules</code> , <code>autoConsumptionSessions</code>
Integra	Push (notificação de promo)
Syntropy	Sugestão de promoção baseada em dados
Mission Control	Promoções ativas, impacto

EVENTOS

CRUD de eventos, configuração, pontos de venda.

Camada	Arquivos
Frontend	pages/AdminEvents.tsx , pages/EventCreator.tsx
Backend	routers/events.ts
Banco	events , salePoints , tablesQr , companies
Integra	Syntropy Planner (onboarding conversacional)
Syntropy	Planejamento, forecast
Mission Control	Status do evento

ONBOARDING / SYNTROPY PLANNER

Criação de evento via conversa com IA.

Camada	Arquivos
Frontend	pages/Onboarding.tsx , components/AIChatBox.tsx
Backend	routers/syntropyPlanner.ts , routers/eventPlanner.ts
Banco	events (criação)
Integra	LLM (invokeLLM)
Syntropy	Planejamento conversacional

SEGURANÇA

Auth, MFA, sessões, auditoria.

Camada	Arquivos
Frontend	<code>pages/Login.tsx</code> , <code>pages/Security.tsx</code>
Backend	<code>routers/auth.ts</code> , <code>_core/oauth.ts</code> , <code>_core/cookies.ts</code>
Banco	<code>users</code> , <code>mfaSecrets</code> , <code>loginSessions</code> , <code>auditLogs</code> , <code>securityAlerts</code>
Integra	Manus OAuth, VAPID (push)

PUSH NOTIFICATIONS

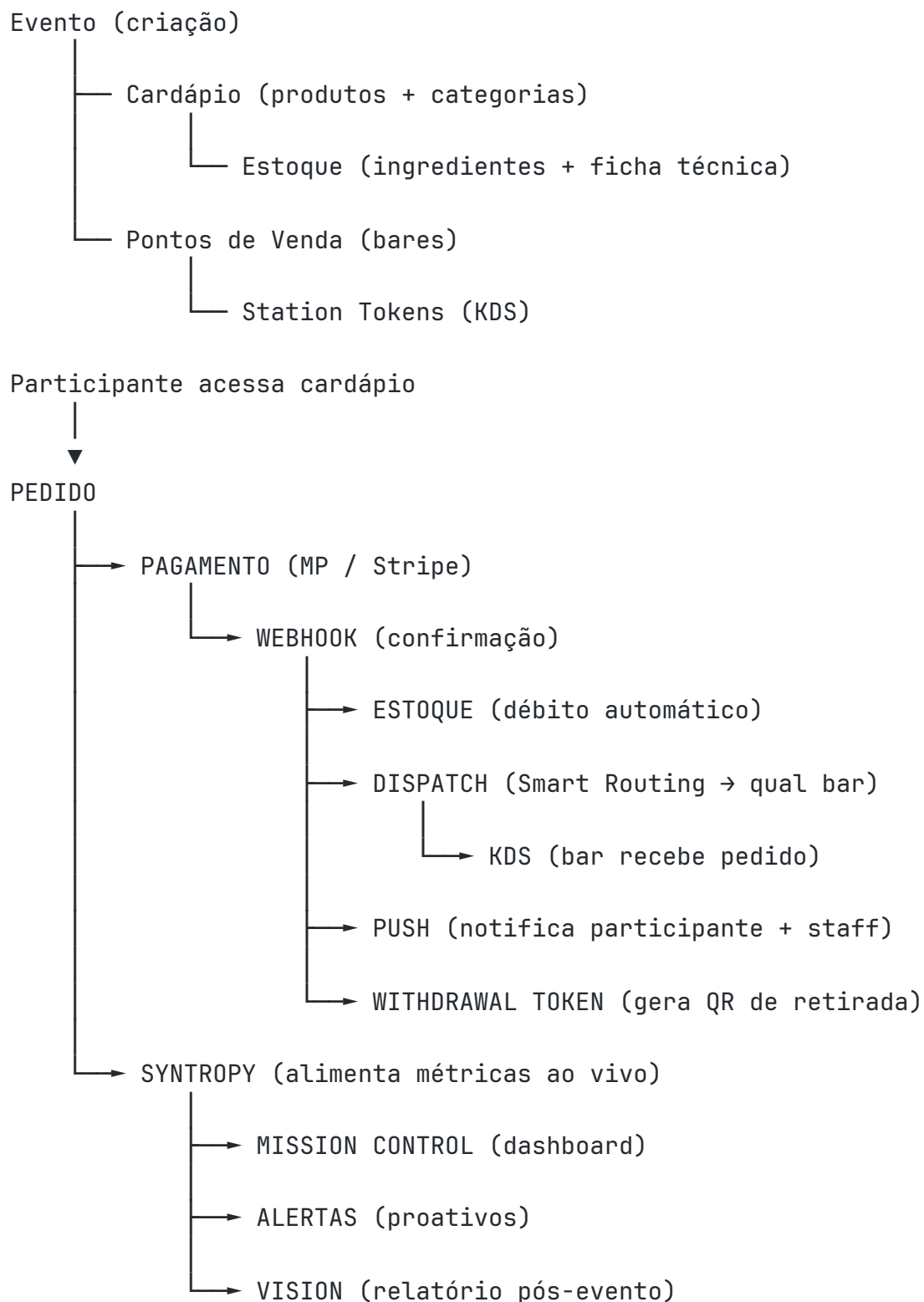
Notificações para participantes e staff.

Camada	Arquivos
Frontend	<code>hooks/usePushNotifications.ts</code> , Service Worker (<code>sw.js</code>)
Backend	<code>pushNotification.ts</code>
Banco	<code>pushSubscriptions</code> , <code>staffPushSubscriptions</code>
Integra	VAPID/Web Push API

Dependency Graph

Como os módulos dependem uns dos outros. Se mexer em um, saiba o que pode quebrar downstream.

Fluxo Principal (cadeia crítica)



Matriz de Dependências

Módulo	Depende de	É dependência de
Pedidos	Produtos, Pagamentos, Estoque	KDS, Push, Syntropy, Mission Control, Vision
Pagamentos	MP API, Stripe API	Pedidos (confirmação), Split, Financeiro
Webhooks	MP/Stripe (externo)	Pedidos, Estoque, Dispatch, Push
Estoque	Produtos, Ingredientes	Syntropy (forecast), Mission Control
Dispatch	Pedidos, Pontos de Venda	KDS, Push
KDS	Dispatch, Station Tokens	Push (notifica participante)
Push	VAPID keys, Subscriptions	— (endpoint final)
Syntropy	Pedidos, Estoque, Eventos, Weather	Mission Control, Alertas, Vision
Mission Control	Syntropy, Pedidos, Estoque, Filas	— (leitura apenas)
Vision	Syntropy, Pedidos, Estoque, CRM	— (relatório final)
Ingressos	Pagamentos	Check-in, CRM
Split	Pagamentos, Stripe Connect	Financeiro
CRM	Pedidos, Ingressos, Perfis	Syntropy, Push

Cadeia de Impacto por Evento

1. Webhook MP falha
 - └ Pedido não confirma
 - └ Estoque não debita (pode vender sem ter)
 - └ KDS não recebe pedido
 - └ Participante não sabe que pagou
 - └ Syntropy conta errado
 - └ Mission Control mostra dados incorretos
2. Dispatch quebra
 - └ Pedido confirmado mas não vai pro bar
 - └ KDS vazio
 - └ Participante espera infinitamente
 - └ Fila cresce sem alerta
3. Estoque não debita
 - └ Produto aparece disponível quando não está
 - └ Syntropy não detecta ruptura
 - └ Vendas continuam até ruptura física
4. Push para
 - └ Staff não sabe de novo pedido (KDS visual resolve)
 - └ Participante não sabe que pedido ficou pronto
 - └ Produtor não recebe alertas Syntropy

Regra de Ouro

Quanto mais à esquerda no fluxo, maior o impacto de uma falha.

Webhook > Pedido > Dispatch > KDS > Push

Se o webhook falha, TUDO downstream para. Se o push falha, o evento continua (só perde notificação).

Impact Map

Altere um arquivo. O que pode quebrar?

Consulte antes de mexer em qualquer arquivo crítico.

Arquivos de Impacto MÁXIMO (●)

`server/routers/orders.ts`

Se alterar, impacta:

- ☒ Pagamentos (criação de payment)
- ☒ Estoque (débito no momento do pedido)
- ☒ KDS (pedido não chega no bar)
- ☒ Push (notificação de confirmação)
- ☒ Dispatch (Smart Routing)
- ☒ Withdrawal (token de retirada)
- ☒ Mission Control (métricas ao vivo)
- ☒ Syntropy (dados de análise)
- ☒ Vision (relatório pós-evento)
- ☒ Split (cálculo financeiro)

Testes obrigatórios: `payment-flow` , `mercadopago` , `orders`

`server/_core/index.ts` (webhooks)

Se alterar, impacta:

- ☒ Confirmação de pagamento (MP + Stripe)
- ☒ Estoque (débito pós-pagamento)
- ☒ Dispatch (routing pós-confirmação)
- ☒ Push (todas as notificações de pagamento)
- ☒ KDS (pedidos não chegam)
- ☒ Mission Control (faturamento incorreto)

Testes obrigatórios: payment-flow , mercadopago , stripe

server/mercadopago.ts

Se alterar, impacta:

- ☒ Criação de Pix (QR Code)
- ☒ Criação de pagamento cartão MP
- ☒ Verificação de pagamento
- ☒ Webhook handling

Testes obrigatórios: mercadopago , payment-flow

server/dispatch.ts

Se alterar, impacta:

- ☒ Smart Routing (pedido → bar correto)
- ☒ KDS (qual estação recebe)
- ☒ Push para staff (notificação de novo pedido)
- ☒ Tempo de fila (routing ineficiente)

Testes obrigatórios: dispatch , smart-routing

Arquivos de Impacto ALTO (🟡)

server/intelligence-engine.ts

Se alterar, impacta:

- ☒ Alertas proativos
- ☒ Sugestões automáticas
- ☒ Ações autônomas
- ☒ Mission Control (feed Syntropy)
- ☒ Vision (relatório pós-evento)

Testes obrigatórios: intelligence-engine , intelligence

server/routers/stationTokens.ts

Se alterar, impacta:

- ☒ Autenticação de estações KDS
- ☒ Bar não recebe pedidos (se token falha)

Testes obrigatórios: station-tokens

server/routers/tickets.ts

Se alterar, impacta:

- ☒ Compra de ingressos
- ☒ Check-in no evento
- ☒ Contagem de público

Testes obrigatórios: tickets

server/routers/withdrawal.ts

Se alterar, impacta:

- ☒ Retirada de itens no bar
- ☒ QR de retirada
- ☒ Confirmação de entrega

Testes obrigatórios: withdrawal, offline-withdrawal

server/routers/ingredients.ts

Se alterar, impacta:

- ☒ Ficha técnica
- ☒ Movimentações de insumos
- ☒ Forecast de ruptura (Syntropy)
- ☒ Mission Control (estoque)

Testes obrigatórios: ingredients

server/routers/stripe.ts

Se alterar, impacta:

-  Pagamento com cartão (Stripe)
-  Express Checkout (Apple/Google Pay)
-  Webhook Stripe

Testes obrigatórios: stripe , payment-flow

Arquivos de Impacto MÉDIO (🟡)

Arquivo	Impacta
server/routers/fluxaControl.ts	Mission Control, CRM, Financeiro (leitura)
server/routers/events.ts	Criação de eventos, pontos de venda, contratos
server/routers/products.ts	Cardápio, estoque de produtos
server/routers/stripeConnect.ts	Split, repasse para produtor
server/pushNotification.ts	Todas as notificações push
server/scheduled-ia-analysis.ts	Análise periódica Syntropy
client/src/pages/Menu.tsx	Experiência de compra inteira
client/src/contexts/CartContext.tsx	Carrinho, cálculo de total

Arquivos de Impacto BAIXO (●)

Arquivo	Impacta
<code>server/routers/syntropyPlanner.ts</code>	Onboarding conversacional
<code>server/routers/pilot.ts</code>	Programa piloto
<code>server/routers/plans.ts</code>	Planos de assinatura
<code>client/src/pages/Home.tsx</code>	Landing page
<code>client/src/pages/Admin*.tsx</code>	Páginas admin (não afeta operação)

Como Usar Este Documento

1. Vai alterar um arquivo? Procure ele aqui.
2. Veja o que impacta.
3. Rode os testes listados.
4. Se impacta pagamento ou estoque → teste manualmente também.
5. Se não está listado aqui → impacto provavelmente baixo, mas rode `pnpm test` mesmo assim.

Ownership

Quem é responsável por cada domínio. Quando crescer, cada pessoa sabe o que é seu.

Ownership Atual

Domínio	Owner	Backup
Produto / Visão	Nanda	—
Syntropy (IA)	Nanda + Manu	—
Pagamentos	Manu	—
Mission Control	Manu	—
Estoque	Manu	—
KDS / Operação	Manu	—
Landing / Marketing	Manu	—
Ingressos	Manu	—
Financeiro / Split	Manu	—
Segurança	Manu	—
Infraestrutura	Manus Platform	—

Ownership Futuro (quando entrar mais gente)

Domínio	Owner ideal	Perfil
Pagamentos	Dev Backend Sr	Experiência com gateways, webhooks, concorrência
Syntropy	Dev IA / ML	Python + LLM + dados de eventos
Mission Control	Dev Frontend Sr	React, real-time, data viz
Estoque	Dev Fullstack	Lógica de negócio, ficha técnica
Mobile / PWA	Dev Mobile	React Native ou PWA avançado
Infraestrutura	DevOps	Deploy, monitoramento, scaling
Design	Designer de Produto	UX de operação (não marketing)

Regras de Ownership

1. **Owner decide** — Se há conflito sobre como implementar algo no domínio, o owner tem a palavra final.
2. **Owner revisa** — Qualquer PR que toque no domínio precisa de review do owner.
3. **Owner documenta** — Se algo muda no domínio, o owner atualiza o DOMAIN_MAP.
4. **Owner testa** — O owner é responsável por garantir que os testes do domínio passam.
5. **Backup existe** — Se o owner sair de férias, o backup assume. Se não tem backup, ninguém mexe.

Escalation

Situação	Quem decide
Bug em pagamento	Owner de Pagamentos
Conflito entre domínios	Nanda (produto)
Decisão de arquitetura	CTO (quando houver) / Nanda
Prioridade de feature	Nanda
Decisão de Syntropy	Nanda

Implementation Kanban

Syntropy Architecture v1.0 — Status de Implementação

Documentação congelada. A partir daqui, só muda quando houver código funcionando.

Motores Cognitivos

Motor	Documentado	Código	Testado	Produção	Próximo passo
Decision Engine	✓	✓	◆	◆	Testes de cobertura completa
Inference Engine	✓	✓	◆	◆	Validar com dados reais
Learning Engine	✓	◆	✗	✗	Implementar consolidação cross-evento
Memory Engine	✓	◆	✗	✗	Implementar decay e consolidação
Context Engine	✓	◆	✗	✗	Implementar adaptação por tipo de evento
Confidence Engine	✓	✗	✗	✗	Implementar score de confiança
Goal Engine	✓	✗	✗	✗	Implementar hierarquia de objetivos
Ethics Engine	✓	◆	✗	✗	Formalizar limites no código
Simulation Engine	✓	✗	✗	✗	Implementar simulação de cenários
Reflection Engine	✓	✗	✗	✗	Implementar autoavaliação pós-evento
Persona Engine	✓	◆	✗	✗	Padronizar tom nos prompts

Motor	Documentado	Código	Testado	Produção	Próximo passo
Communication Engine	✓	◆	✗	✗	Implementar adaptação por audiência
Evolution	✓	✗	✗	✗	Roadmap de longo prazo

Legenda:

- ✓ Completo
 - ◆ Parcial (existe algo mas não cobre o documento inteiro)
 - ✗ Não existe
-

Funcionalidades Operacionais

Feature	Documentado	Código	Testado	Produção	Próximo passo
Smart Routing	✓	✓	✓	✓	Otimizar com dados reais
Forecast de demanda	✓	✓	✖	✖	Validar acurácia
Alertas proativos	✓	✓	✖	✖	Reduzir falsos positivos
Vision (relatório)	✓	✓	✖	✖	Adicionar Reflection
Promoções automáticas	✓	✖	✖	✖	Syntropy sugerir automaticamente
Previsão climática	✓	✓	✖	✖	Integrar no planejamento
Comunicação pós-evento	✓	✖	✖	✖	Implementar
Precificação dinâmica	✓	✖	✖	✖	Implementar

Infraestrutura

Feature	Documentado	Código	Testado	Produção
Pedidos	✓	✓	✓	✓
Pagamento Pix (MP)	✓	✓	✓	✓
Pagamento Cartão (MP)	✓	✓	✓	✓
Pagamento Cartão (Stripe)	✓	✓	✓	◆ (sandbox)
Webhooks	✓	✓	✓	✓
KDS	✓	✓	✓	✓
Estoque (produtos)	✓	✓	✓	✓
Estoque (insumos)	✓	✓	✓	✓
Ingressos	✓	✓	✓	✓
Check-in	✓	✓	◆	◆
Retirada QR	✓	✓	✓	✓
Push Notifications	✓	✓	✓	✓
Offline Mode	✓	✓	✓	✓
Split (Stripe Connect)	✓	✓	◆	◆
Contratos digitais	✓	✓	◆	◆
MFA (2FA)	✓	✓	✓	✓
Mission Control	✓	✓	◆	✓
Onboarding (Syntropy)	✓	✓	◆	✓
CRM	✓	✓	◆	◆

Prioridade de Implementação (próximos sprints)

#	O que	Por quê	Impacto
1	Confidence Engine	Sem isso, Syntropy age sem medir certeza	Alto
2	Reflection Engine	Sem isso, não melhora entre eventos	Alto
3	Promoções automáticas	Receita direta para o produtor	Alto
4	Memory Engine (consolidação)	Sem isso, cada evento começa do zero	Médio
5	Communication Engine (adaptação)	Melhora UX para todos os perfis	Médio
6	Simulation Engine	Produtor pode testar cenários antes	Médio
7	Comunicação pós-evento	Retenção de participantes	Médio
8	Precificação dinâmica	Otimização de receita	Baixo (futuro)

Regra

Nenhum motor novo é documentado até que os existentes estejam pelo menos em  no código.

Documentação congelada. Próximo passo = código.

Versão

Syntropy Architecture v1.0 – FROZEN

Data: 29 de junho de 2026

Próxima revisão: Após primeiro evento real com Confidence + Reflection im

Syntropy Architecture v1.0 – FROZEN

Data de congelamento: 29 de junho de 2026

Próxima revisão permitida: Após primeiro evento real com Confidence + Reflection implementados

O que está congelado

Toda a documentação da pasta `docs/` está congelada nesta versão. Isso significa:

1. **Não criar novos motores** até que os existentes estejam implementados
2. **Não alterar documentos existentes** exceto para corrigir erros factuais
3. **Não expandir escopo** sem evidência de evento real

O que pode mudar

1. **IMPLEMENTATION_KANBAN.md** — Atualizar status conforme código avança
2. **OWNERSHIP.md** — Atualizar quando entrar gente nova
3. **Correções factuais** — Se um documento diz algo que o código contradiz

Motivo do congelamento

A documentação está completa o suficiente para:

- Onboarding de qualquer dev em ~1 hora
- Guiar implementação de todos os motores
- Proteger fluxos críticos
- Definir identidade e comportamento da Syntropy

O risco agora é documentar mais rápido do que implementar. O valor passa a vir de **código funcionando em eventos reais**.

Próximo capítulo

Execution v1.0

Implementar os motores documentados, medir em eventos reais, e só então evoluir a arquitetura.

Inventário Final

Categoria	Documentos	Status
Visão do Produto	7	✓ Congelado
Arquitetura Técnica	10	✓ Congelado
Syntropy (Motores)	15	✓ Congelado
Setup & Regras	3	✓ Congelado
Operacional	3	↻ Atualizável (Kanban, Ownership, Version)
Total	38 documentos	